CLOUD COMPUTING AND BIG DATA

ASSIGNMENT 2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

Student Details	Information
Student Name	G Rahul
Registration Number	192572008
Course	Cloud Computing and Big Data
Course Outcome	CO2 – Analyze virtualization architectures to identify performance and security issues in cloud computing environments
Submission Date	6/11/2026

Important Note About the Practical Sections

This document provides the architecture analysis, mathematical model, security evaluation, proposed test procedure and final design. Tasks 8–10 require actual experiments. Since no benchmark measurements or screenshots were supplied, this assignment does not invent experimental results. The tables in Tasks 8–10 are clearly marked for the student's real measurements. Run the commands on the available machine, insert the measured values and screenshots, and then complete the comparison.

The numerical overhead percentages used in the analytical examples are stated modelling assumptions for comparison, not measured facts or universal benchmark values.

Introduction

EduCloud is a university private-cloud platform containing 16 physical hosts. Each host has 64 CPU cores, 512 GB RAM, NVMe SSD storage and two 25 Gbps network interface cards. Around 400 virtual machines support the Student Information System (SIS), Learning Management System (LMS), examination portal and research workloads. The platform experiences slow responses and failed logins during semester registration week. The audit also identified shared physical hosts across departments, outdated guest kernels and uncontrolled copying of VM images.

The aim of this assignment is to compare three different virtualization approaches, model their performance, examine security risks and recommend an architecture. The design is also evaluated for external Spark/Hadoop tenants that run long, I/O-intensive jobs alongside sensitive student systems.

System Parameters and Modelling Assumptions

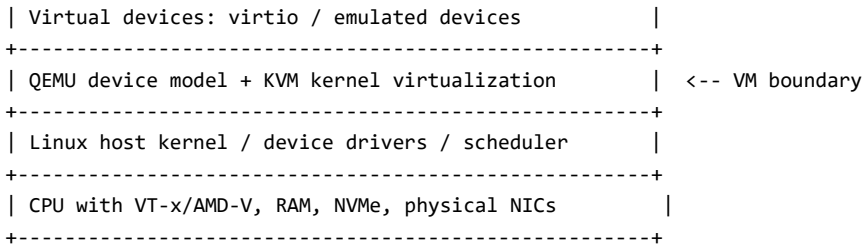
Parameter	Given value / assumption	Use in this report
Physical CPU cores	64 cores per host	CPU capacity model
RAM	512 GB per host	Memory capacity model
Average VM memory demand	8 GB	VM density estimate
vCPU:pCPU ratios	1:1, 2:1, 4:1, 8:1	CPU overcommit analysis
Cost of one VM exit	1.5 microseconds	KVM/Kata event-cost illustration
Native system-call cost	0.2 microseconds	Container system-call illustration
Network line rate	25 Gbps per NIC	Network sharing model
Host OS / hypervisor reserve	32 GB RAM	Assumption; leaves 480 GB for workloads
Example VM size	2 vCPU and 8 GB RAM	Assumption for density calculations
Illustrative CPU overhead	KVM 3%; Docker 2%; Kata 5%	Assumptions only; validate experimentally
Illustrative I/O efficiency	KVM 90%; Docker 95%; Kata 88%	Assumptions only; compare relative paths

TASK 1 – Characterise the Virtualization Architectures (5 Marks)

The three selected approaches are architecturally different: KVM/QEMU full virtual machines, Docker OS-level containers and Kata Containers using lightweight virtual machines. KVM and Kata provide VM-based isolation, while Docker provides container-based isolation.

1.1 Type-1-style KVM/QEMU full virtualization

```
+-----------------------------------------------------+
| Applications: SIS / LMS / Examination Portal        |
+-----------------------------------------------------+
| Guest libraries and guest operating system          |
+-----------------------------------------------------+
```

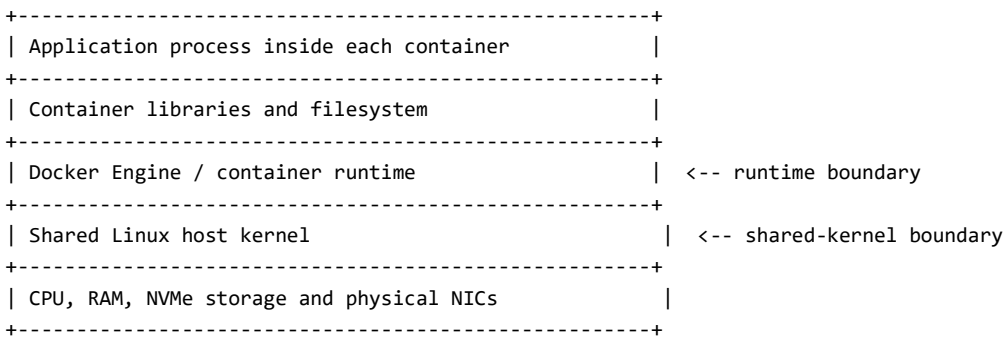


KVM uses hardware virtualization extensions in the processor. The Linux kernel provides the KVM virtualization interface, while QEMU supplies the virtual machine process and device models. Each VM runs its own guest operating system. The hypervisor and hardware memory-translation mechanisms enforce the main VM boundary.

- CPU: guest vCPUs are scheduled onto physical CPU cores. Hardware virtualization handles privileged guest execution; transitions to the host may cause VM exits.
- Memory: guest physical addresses are translated to host physical addresses using second-level translation such as Intel EPT or AMD NPT.
- Storage: virtual disks are backed by files, logical volumes or block devices; virtio reduces device-emulation overhead.
- Network: virtual NICs connect through a virtual switch/bridge and then the physical NIC.

Relevance to EduCloud: separate guest operating systems suit existing SIS, LMS and examination applications that require strong workload separation and different operating-system configurations.

1.2 Docker OS-level containers

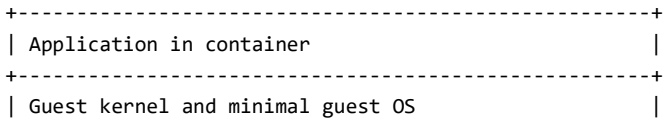


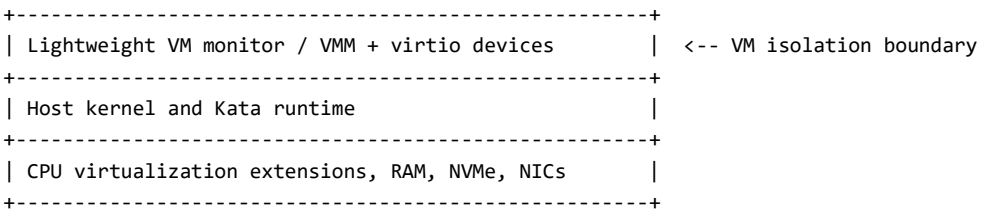
Docker containers package an application and its dependencies but share the host operating-system kernel. Namespaces provide separate views of processes, network interfaces, mounts and users. Control groups (cgroups) limit and account for CPU, memory and I/O. Linux capabilities, seccomp and mandatory access-control profiles can reduce privileges.

- CPU: processes are scheduled by the host kernel and constrained through CPU cgroups.
- Memory: cgroup memory limits and accounting control each container's memory use.
- Storage: layered images and writable container layers use the host filesystem; volumes provide persistent data.
- Network: virtual Ethernet interfaces and bridges connect containers; firewall and network policies control traffic.

Relevance to EduCloud: containers are lightweight and start quickly, making them useful for stateless web services, development environments and scalable microservices. However, a shared kernel is a concern for untrusted tenants.

1.3 Kata Containers / lightweight micro-VMs





Kata presents a container-like workflow but runs each workload inside a lightweight virtual machine with its own guest kernel. A virtual machine monitor and hardware virtualization enforce the boundary. This provides stronger separation than ordinary shared-kernel containers, although it generally costs more resources and startup time.

- CPU: each lightweight VM receives virtual CPUs scheduled by the host.
- Memory: guest memory is backed by host memory and protected by hardware-assisted address translation.
- Storage: container image content is made available to the guest through a virtualized storage path.
- Network: virtual NICs connect through the host networking layer; virtual switching and packet processing remain relevant.

Relevance to EduCloud: Kata is a useful isolation layer for external research collaborators because each workload receives a separate guest kernel while retaining a container-oriented deployment model.

Approach	Isolation enforcement	Main strength	Main limitation
KVM/QEMU VM	Hardware virtualization, hypervisor and guest boundary	Mature VM separation and independent guest OS	More memory and device-virtualization overhead
Docker container	Namespaces, cgroups, capabilities, seccomp and shared kernel	High density and fast startup	Shared-kernel attack surface
Kata micro-VM	Hardware VM boundary plus lightweight guest kernel	Stronger tenant separation than ordinary containers	Higher overhead than standard containers

TASK 2 – Identify the Dominant Overhead Source (2 Marks)

Approach	Dominant overhead	Where it occurs and why it matters
KVM/QEMU	VM exits and virtual I/O processing	A guest operation that requires hypervisor handling can leave guest execution. Device emulation and interrupt handling add work. The cost rises with privileged events and I/O rate.
Docker	Shared-kernel scheduling, system-call and virtual-switch processing	Container system calls enter the host kernel directly, but isolation checks, networking and contention are still processed there. More containers and packets increase scheduling and switching work.
Kata	VM exits plus virtio and host/guest I/O transitions	The workload crosses a guest-kernel and virtual-device boundary. More I/O operations create more transitions and virtual-device work; more micro-VMs also increase host scheduling pressure.

For KVM, VM exits are an appropriate focus because the supplied assignment gives an assumed cost of 1.5 microseconds per exit. If a workload causes E exits per second, the approximate CPU time spent handling exits is E × 1.5 microseconds per second. For Docker, the supplied native system-call cost is 0.2 microseconds; container system calls are not automatically all extra-cost calls, so the relevant comparison is the additional isolation, scheduling and networking work rather than charging every call as overhead. Kata has both VM-related transitions and virtual I/O processing.

As co-located instances increase, contention can become more important than the isolated cost of one event. CPU run queues become longer, memory pressure increases, and the virtual switch processes more traffic. Under an I/O storm, queues and interrupts can increase tail latency even if average CPU utilisation appears acceptable.

TASK 3 – Build a Quantitative Performance Model (6 Marks)

3.1 CPU capacity and overcommit

Let C = 64 physical cores, R = vCPU:pCPU overcommit ratio, V = number of assigned vCPUs, and h = assumed virtualization overhead fraction. The total assigned vCPU capacity is V = 64R. The nominal physical-core share per assigned vCPU is 1/R. A simple effective capacity per assigned vCPU is:

Effective capacity per vCPU = (1/R) × (1 − h) physical-core equivalents.

For an example 2-vCPU VM, its ideal scheduled share is 2/R cores, before overhead and steal time. Therefore:

Effective capacity per 2-vCPU VM = (2/R) × (1 − h − s), where s is the additional fraction lost to steal time or contention. This is a simplified model; h and s must be measured separately in a real test.

Overcommit	Total assigned vCPUs	Ideal capacity per vCPU	Ideal capacity per 2-vCPU VM
1:1	64	1.00 core	2.00 cores
2:1	128	0.50 core	1.00 core
4:1	256	0.25 core	0.50 core
8:1	512	0.125 core	0.25 core

Illustrative KVM example at 2:1: with h = 0.03 and s = 0, one 2-vCPU VM receives (2/2) × 0.97 = 0.97 physical-core equivalents. At 4:1 it receives (2/4) × 0.97 = 0.485 core equivalents. This does not mean the VM is permanently pinned to those cores; it is an average capacity model.

3.2 Memory capacity

The host has 512 GB RAM. Reserving 32 GB for the host, hypervisor and management services leaves 480 GB for workloads. With an average VM demand of 8 GB:

Maximum VMs before memory overcommit = floor(480 GB / 8 GB) = 60 VMs per host.

If the reserve is ignored, the theoretical value is 512/8 = 64 VMs. The practical estimate of 60 is safer. When demand exceeds available memory, ballooning, swapping or page sharing may occur. Swapping can sharply increase latency, especially for database and interactive SIS services.

3.3 I/O and network throughput

The network line rate is 25 Gbps, equal to 3.125 GB/s before protocol overhead. If N active instances share one NIC equally, a simplified per-instance throughput is:

Per-instance throughput = (25 Gbps / N) × η, where η is the assumed efficiency of the architecture and network path.

Approach	Illustrative η	N = 10: aggregate share per instance	N = 40: aggregate share per instance
KVM/QEMU	0.90	2.25 Gbps	0.5625 Gbps
Docker	0.95	2.375 Gbps	0.59375 Gbps
Kata	0.88	2.20 Gbps	0.55 Gbps

These are equal-share estimates, not guaranteed bandwidth. Real throughput depends on packet size, CPU availability, NIC queues, virtual-switch design, storage limits and traffic patterns. For storage, use the same form: per-instance storage throughput = (measured physical storage throughput / number of active I/O

instances) × storage-path efficiency. The assignment does not provide a numeric NVMe throughput, so a numeric storage result cannot be derived without measurement.

3.4 Density under a 10% degradation target

A host can meet a 10% performance-degradation target only if the combined virtualization overhead and contention loss remain at or below 10%. A simple constraint is h + s ≤ 0.10. Memory also limits density to 60 example VMs per host. For 2-vCPU VMs, the CPU assignment limit is 32, 64, 128 or 256 VMs at 1:1, 2:1, 4:1 or 8:1 respectively. Combining CPU and memory limits gives:

Ratio	CPU-limited 2-vCPU VMs	Memory limit	Maximum by these two limits
1:1	32	60	32
2:1	64	60	60
4:1	128	60	60
8:1	256	60	60

The table gives resource-allocation ceilings, not proof that all instances meet the 10% service target. At 4:1 and 8:1, scheduler contention and steal time may violate the target before memory is exhausted. A conservative production starting point is 1:1 or 2:1 for latency-sensitive SIS workloads, followed by measurement-based adjustment. Docker containers may achieve higher instance density when their actual memory and CPU limits are smaller than a full 8 GB VM; that benefit cannot be quantified fairly without a container memory-demand measurement.

3.5 Comparing the illustrative architecture costs

Approach	Assumed CPU overhead h	Effective 2-vCPU VM capacity at 2:1, s=0	Interpretation
KVM/QEMU	3%	0.970 core equivalents	Moderate VM overhead with independent guest OS
Docker	2%	0.980 core equivalents	Low assumed runtime overhead; shared kernel risk
Kata	5%	0.950 core equivalents	Higher assumed cost for stronger VM boundary

The overhead values are modelling assumptions chosen to demonstrate the calculation. They must be replaced or calibrated using Task 9 results.

TASK 4 – Analyse Performance Under Different Operating Conditions (4 Marks)

Condition	KVM/QEMU	Docker	Kata
Best case	CPU-bound VMs with adequate reserved cores, virtio devices and low oversubscription. Few exits and little I/O emulation keep latency stable.	Small stateless services with low memory use and little cross-container traffic. Shared-kernel scheduling avoids a separate guest OS per workload.	Small isolated workloads with low I/O rates. Lightweight guest VMs avoid much of the cost of full general-purpose VMs.
Typical registration load	Web requests and database activity share CPU, storage and network queues. Overcommit increases scheduling delay; database I/O can raise response time.	Many web containers may be dense, but all compete in the same host kernel. CPU, memory and bridge traffic need cgroup limits and monitoring.	Stronger isolation helps separate workloads, but guest/host I/O transitions and additional memory use can reduce density during busy periods.
Worst case	Noisy-neighbour CPU use, memory overcommit, swapping or live migration during peak load increases steal time, VM exits and I/O	A noisy container can consume shared CPU, memory or network resources. A kernel/runtime vulnerability has a wider	Many busy micro-VMs plus heavy I/O create virtual-device and host scheduling pressure. It is more isolated than Docker but can have

	queueing. Login latency and timeout rates rise.	impact because containers share the host kernel.	higher latency and lower density.

The model predicts that memory pressure is especially dangerous for the SIS because swapping can turn a short storage operation into a much slower one. The 3× registration-week peak should therefore be handled with reserved capacity, admission control, database tuning, caching and separate resource pools rather than relying on high CPU overcommit.

TASK 5 – Security Threat and Vulnerability Analysis (3 Marks)

Threat category	KVM/QEMU exposure	Docker exposure	Kata exposure	Possible CIA impact
VM/container escape	Hypervisor, QEMU or device-model bug may allow escape from a guest.	Container-runtime or kernel vulnerability can cross the shared-kernel boundary.	VMM or guest/host integration bug may permit escape; VM boundary reduces some shared-kernel risk.	Confidentiality and integrity breach; host availability loss.
Cross-tenant side channel	Co-resident VMs may share CPU caches and other hardware resources.	Containers share the host kernel and hardware, so co-resident processes may observe timing or resource effects.	Separate guest kernels help, but shared CPU caches and host hardware remain.	Potential information leakage, including sensitive data.
Unpatched guest/runtime	Outdated guest kernels and hypervisor components increase attack opportunities.	Host kernel and runtime patching are critical; all containers depend on the host kernel.	Both host components and guest kernels need patching.	Integrity compromise and service disruption.
Image tampering / VM sprawl	Copied VM images may contain old packages, secrets or insecure settings.	Untrusted or unsigned images may contain malicious layers or vulnerable dependencies.	Container image and micro-VM guest image supply chain both require control.	Confidentiality and integrity loss; availability impact.
Virtual network sniffing/spoofing	Misconfigured virtual switches or flat networks can expose tenant traffic.	Bridge networks and weak network policies can allow unwanted container-to-container traffic.	Virtual NIC and host network configuration can expose traffic if segmentation is weak.	Student-record disclosure or service impersonation.
Insecure live migration	Unencrypted migration traffic or weak management access can expose VM state.	Container relocation and registry/control-plane credentials can be abused.	Migration and guest memory/state transfer require protection where supported.	Confidentiality and integrity loss.

For EduCloud, the most urgent risks are shared physical hosts across departments, outdated guest kernels and uncontrolled image copying. Student records require confidentiality, integrity and availability. The platform should use tenant identity, host-pool separation for sensitive systems, signed and scanned images, patch compliance, encrypted management/migration traffic, and default-deny virtual-network policies.

TASK 6 – Formulate the Isolation–Performance Trade-off (3 Marks)

Let $x(a,t)$ be the number of instances of architecture a assigned to tenant t; let $y(a,t)$ indicate the selected architecture; let H_{sis} be hosts reserved for SIS; and let R be the CPU overcommit ratio. A practical objective is to minimise operating cost and performance degradation while meeting the required isolation level.

Illustrative decision function:

Minimise $J = \alpha \cdot C_{host} + \beta \cdot D_{perf} + \gamma \cdot R_{security}$

where C_host is the number/cost of allocated hosts, D_perf is measured percentage performance degradation, and R_security is a weighted residual-risk score. α, β and γ are university-selected weights. For SIS, γ and β should be high because student-record protection and login response are critical.

Subject to:

- D_perf(SIS) ≤ 10% during the defined service-level test.
- SIS instances run only on approved dedicated or strongly isolated host pools.
- Memory demand per host must remain within the reserved workload memory.
- CPU assignment and measured steal time must remain within the approved limit.
- External research workloads must not share the SIS trust boundary; network policies must deny access to SIS data networks.
- Total allocated hosts must not exceed the 16-host capacity unless additional hosts are approved.

Decision: use KVM-based VMs for SIS/LMS and Kata micro-VMs for external collaborators. Standard Docker containers can be used for trusted internal stateless services, but not as the only isolation boundary for untrusted external tenants.

TASK 7 – Evaluate Mitigation Strategies (4 Marks)

Mitigation	Threat addressed and benefit	Residual risk	Performance/density cost and SIS decision
Dedicated host pool for SIS	Reduces noisy-neighbour interference and prevents external research workloads from sharing SIS physical hosts.	Hypervisor, administrator, firmware and network risks remain.	Uses more hosts and reduces overall density. Acceptable for SIS because predictable response and tenant separation are priorities.
Signed, scanned and versioned images	Reduces image tampering, uncontrolled copies and deployment of known vulnerable images.	A signed image can still contain vulnerable software; signing-key compromise remains possible.	Small deployment and scanning overhead; little runtime cost. Strongly acceptable.
Patch guest and host kernels on a schedule	Reduces exposure to known kernel, hypervisor and runtime vulnerabilities.	Zero-day vulnerabilities and delayed patches remain possible.	Patching may require maintenance windows or live migration. Schedule outside registration peaks and verify versions.
Micro-segmentation and default-deny network policy	Limits lateral movement and prevents research tenants from reaching SIS services or databases.	Misconfiguration and compromised policy controllers remain possible.	Adds policy processing and administration overhead; acceptable for student-data protection.
Encrypted management and migration traffic	Protects VM memory/state and control traffic from interception or tampering.	Endpoint compromise and credential theft remain possible.	Encryption consumes some CPU/network capacity; test during migration and avoid peak periods.
Disable SMT for highest-sensitivity workloads (where supported)	Can reduce some sibling-thread resource sharing and side-channel exposure.	Does not eliminate all cache or microarchitectural side channels.	May reduce available CPU throughput/density. Consider for the most sensitive SIS database hosts if risk assessment justifies it.

The model-based cost should be validated with measurements. For example, if disabling SMT reduces usable CPU capacity by an assumed 20%, a 64-core host's effective scheduling capacity would be modelled as 51.2 core equivalents. This is only an illustrative scenario; the actual effect depends on the processor and workload. The SIS decision should be based on the measured throughput and security benefit.

TASK 8 – Implement and Test (4 Marks)

The assignment asks for KVM/QEMU, Docker/Podman and Kata or gVisor tests, with approximately 1, 2, 4, 8 and 16 co-located instances. A laptop may not support a full 16-instance VM test. In that case, reduce the scale, record the CPU/RAM limitation and do not claim that the laptop reproduces the university's 64-core server.

8.1 Suggested test environment

- Host: Ubuntu Linux, or Ubuntu running under WSL2 where supported. Record CPU model, physical/logical cores, RAM, kernel version and storage type.
- VM-based: KVM/QEMU. Nested virtualization may be required if Ubuntu itself runs inside another VM.
- Container-based: Docker or Podman.
- Micro-VM: Kata Containers if the machine and kernel support it. If Kata cannot be installed, document the limitation and do not substitute ordinary Docker while labelling it as Kata.
- Use the same workload input, CPU/memory limits, test duration and repetitions for each architecture.

8.2 Example Linux commands

Record the machine configuration:

```
lscpu
free -h
uname -a
lsblk
ip -br link
```

Install common tools on Ubuntu (package names can vary by release):

```
sudo apt update
sudo apt install -y sysbench fio iperf3 stress-ng sysstat
```

CPU test example (run inside each test instance):

```
sysbench cpu --cpu-max-prime=20000 --threads=2 --time=60 run
```

Memory test example:

```
stress-ng --vm 1 --vm-bytes 1G --timeout 60s --metrics-brief
```

Disk test example (use a dedicated test file; never point fio at a real disk device containing data):

```
fio --name=educloud-disk --filename=./fio-test.bin --size=1G \
  --rw=randrw --bs=4k --iodepth=16 --runtime=60 --time_based \
  --direct=1 --group_reporting
```

Network test example (run iperf3 server on one endpoint and client on another):

```
# Server
iperf3 -s

# Client
iperf3 -c SERVER_IP -t 60 -P 4
```

Docker instance example:

```
docker run -d --name test1 --cpus=2 --memory=2g ubuntu:24.04 sleep infinity
docker exec test1 sysbench cpu --cpu-max-prime=20000 --threads=2 --time=60 run
docker stats
```

KVM and Kata commands depend on the installed distribution, image and runtime configuration. Save the exact VM creation command or configuration file, guest image name, vCPU count, RAM size and virtual NIC/storage settings. Do not compare a 1-vCPU container with a 4-vCPU VM.

8.3 Test matrix and security validation

Test	Instance counts	Measurements / evidence
CPU-bound	1, 2, 4, 8, 16 (or feasible subset)	Sysbench events/sec, CPU utilisation, steal time

Memory-intensive	1, 2, 4, 8, 16	Memory used, paging/swapping, completion time
Disk I/O	1, 2, 4, 8, 16	IOPS, MB/s, average and p95 latency
Network	1, 2, 4, 8, 16	Throughput, retransmissions, latency if measured
Noisy neighbour	One benchmark instance plus one stress instance	Change in benchmark throughput and p95 latency
Security check	At least one instance per approach	Namespaces/cgroups, guest-kernel separation, network policy or mitigation status

Docker security inspection examples:

```
docker inspect test1
docker exec test1 cat /proc/1/cgroup
docker exec test1 readlink /proc/1/ns/pid
```

For the noisy-neighbour test, first measure the benchmark alone. Then run a CPU or I/O stress workload in a second instance while repeating the same benchmark. Record the percentage change. Keep the same number of benchmark threads and resource limits.

Safety: fio writes a test file and stress tools deliberately load the machine. Use a test directory, monitor temperatures and stop if the system becomes unstable.

TASK 9 – Record Experimental Performance (3 Marks)

Fill in the following tables with actual output from the tests. Run each test at least three times. Report mean and standard deviation. Do not copy the illustrative assumptions from Task 3 into the measured-results column.

9.1 Raw measurement table

Approach	Instances	CPU throughput (events/s)	Disk (MB/s)	Network (Gbps)	Mean latency (ms)	p95 latency (ms)
KVM	1	Enter result	Enter result	Enter result	Enter result	Enter result
KVM	2 / 4 / 8 / 16	Enter result	Enter result	Enter result	Enter result	Enter result
Docker	1	Enter result	Enter result	Enter result	Enter result	Enter result
Docker	2 / 4 / 8 / 16	Enter result	Enter result	Enter result	Enter result	Enter result
Kata	1	Enter result	Enter result	Enter result	Enter result	Enter result
Kata	2 / 4 / 8 / 16	Enter result	Enter result	Enter result	Enter result	Enter result

9.2 Repeatability and security mitigation

Approach / condition	Run 1	Run 2	Run 3	Mean	Standard deviation
KVM baseline	—	—	—	Calculate	Calculate
KVM with mitigation	—	—	—	Calculate	Calculate
Docker baseline	—	—	—	Calculate	Calculate
Docker with mitigation	—	—	—	Calculate	Calculate
Kata baseline	—	—	—	Calculate	Calculate

Mean = (Run 1 + Run 2 + Run 3) / 3. Standard deviation describes variation between repeated runs. For each approach, calculate degradation against bare metal or the least-virtualized baseline:

Degradation (%) = ((baseline throughput − measured throughput) / baseline throughput) × 100.

For latency, report the percentage increase separately: ((measured latency − baseline latency) / baseline latency) × 100.

Required graphs: (1) throughput versus number of instances; (2) p95 latency under noisy-neighbour interference. Use measured values only. Include axis labels, units, legend and test conditions.

TASK 10 – Compare Theory with Experiment (4 Marks)

After completing Task 9, compare the measurements with the predictions in Task 3 and Task 4. Use the following structure and replace the prompts with your actual findings.

- Trend agreement: State whether throughput decreased and latency increased as instance count grew. Explain whether this agrees with the CPU, memory and I/O sharing model.
- Scalability: Identify which approach retained the highest throughput and lowest p95 latency at the largest tested instance count. Quote the measured values.
- Prediction error: Calculate absolute error = |measured value − predicted value|. Percentage error = (absolute error / predicted value) × 100, where the predicted value is non-zero.
- Explain differences: Possible causes include host CPU differences, caching, scheduler behaviour, nested virtualization, storage caching, NIC limitations, guest configuration and unmodelled I/O paths.
- Security interpretation: State whether the practical isolation inspection supports the Task 5 comparison. A successful namespace check does not prove that a container is immune to kernel vulnerabilities.

Metric	Task 3 prediction	Task 9 measurement	Difference / explanation
CPU throughput at selected scale	Insert model value	Insert measured mean	Calculate and explain
Disk throughput	Insert model value	Insert measured mean	Calculate and explain
Network throughput	Insert model value	Insert measured mean	Calculate and explain
p95 latency under interference	Expected trend	Insert measured value	Explain noisy-neighbour effect

TASK 11 – Recommend and Defend the Final Architecture (4 Marks)

The recommended architecture is a hybrid design. KVM/QEMU virtual machines should host the SIS, LMS, examination portal and other services that require independent guest operating systems. Kata micro-VMs should be used for external research collaborators' workloads. Standard Docker containers can support trusted internal microservices, build jobs and stateless applications, but should not be the sole isolation boundary for untrusted external tenants.

11.1 Proposed host-pool layout

```
EDUCLOUD PRIVATE CLOUD
|
+-- Management pool
|   +-- Hypervisor management, monitoring, image registry
|
+-- Sensitive university pool (KVM)
|   +-- SIS application VMs
|   +-- SIS database VMs
|   +-- LMS and examination portal VMs
|   +-- Reserved CPU/RAM and restricted admin access
|
+-- Internal application pool
|   +-- Trusted Docker services
|   +-- cgroups, namespaces, signed images, network policies
|
+-- External research pool (Kata micro-VMs)
    +-- Collaborator Spark/Hadoop workloads
    +-- Separate virtual networks and storage policies
```

```
+-- Quotas, dedicated NIC queues where available
```

11.2 Reasons for the recommendation

- Security: external collaborators do not share the SIS host pool or kernel boundary. KVM provides mature VM isolation for university systems; Kata adds a lightweight VM boundary for research containers.
- Performance: SIS hosts can reserve CPU and memory and avoid direct competition with long-running Spark/Hadoop jobs. This reduces noisy-neighbour risk during registration.
- Operations: central image signing, vulnerability scanning, patch management, host monitoring and access control make the environment easier to govern.
- Growth: host pools can be expanded independently. New research capacity can be added without placing external workloads on SIS hosts.

The recommendation is not based on claiming that one technology is always fastest or most secure. It balances SIS data protection, predictable latency, density and operational complexity. The final report should insert Task 9 measurements to confirm the expected performance ranking on the available test machine.

11.3 Operational controls and limitations

- Maintain approved golden images and remove outdated VM/container images.
- Use role-based access, multi-factor authentication for administrators and audited management access.
- Monitor CPU ready/steal time, memory pressure, storage latency, network drops, VM exits where observable and application login latency.
- Schedule patching and live migration outside registration peaks where possible.
- The design requires more operational effort than a single shared container platform and may reduce overall density because sensitive and external workloads use separate pools.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The new requirement introduces external research collaborators and long-running Spark/Hadoop jobs. These jobs are I/O- and network-intensive and may run for hours. Registration-week demand is about three times normal, and live migration is used during patching. A data-protection regulation also requires demonstrable isolation of student records from external tenants.

- The earlier assumption that all tenants are similarly trusted is no longer valid. External collaborators must be treated as less-trusted tenants.
- The workload mix changes from mainly interactive university applications to a combination of latency-sensitive SIS requests and batch data processing.
- Heavy Spark/Hadoop disk and network traffic can fill shared queues, increase storage latency and consume NIC bandwidth.
- A 3× peak increases the risk that an overcommitted host will exceed the 10% degradation target.
- Live migration adds network traffic and transfers VM state. If migration is not protected, memory contents and management credentials may be exposed.

The threat model now gives greater weight to tenant escape, lateral movement, data exfiltration, resource exhaustion and migration interception.

B. Evaluation of the Original Design (2 Marks)

A design that places all VMs and containers in one shared pool is no longer adequate. Even if the average CPU model appears acceptable, the equal-share network and I/O models show that per-instance throughput falls as active workloads increase. Batch jobs can create long queues and cause tail-latency spikes for SIS users.

The original design is adequate only if it already provides strict host and network separation, resource reservations and migration protection. Ordinary Docker containers alone are not sufficient as the isolation boundary for untrusted external collaborators because they share the host kernel. The hybrid design is more appropriate.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the design into separated host pools with stronger isolation for external tenants.

Pool	Workloads	Isolation and resource controls
Sensitive university pool	SIS, SIS database, LMS and examination portal	KVM VMs; dedicated hosts where required; reserved CPU/RAM; strict network segmentation; encrypted management and migration
Internal application pool	Trusted internal web services and microservices	Docker containers with namespaces, cgroups, seccomp, signed images and quotas
External research pool	Spark/Hadoop and collaborator workloads	Kata micro-VMs; separate tenant networks; CPU, memory, storage and bandwidth quotas; separate credentials and data paths
Management pool	Monitoring, image registry and hypervisor administration	Restricted administrative network, role-based access and audit logging

Where supported, use SR-IOV or dedicated NICs/queues for high-throughput research traffic, while retaining network policy enforcement and monitoring. Encrypt live migration traffic, authenticate both endpoints and restrict migration to a management network. Do not allow research workloads to mount SIS storage or access SIS database networks.

Effect on density: separate host pools and micro-VMs reduce maximum packing density and may require additional hosts. Effect on throughput: dedicated capacity and NIC queues reduce contention for SIS and improve predictable performance. Effect on attack surface: the design adds pool-management components but reduces the number of trust relationships between external workloads and sensitive university services.

D. Evidence-Based Defence (2 Marks)

The decision follows from the model and must be confirmed with the experiments. The memory model estimates a practical limit of 60 example 8 GB VMs per 512 GB host after a 32 GB reserve. The CPU model shows that high overcommit ratios increase the risk of steal time, while the network model shows that equal-share throughput decreases as more active instances share a 25 Gbps NIC. These effects become more serious when research workloads run for hours and registration traffic rises to three times normal.

The noisy-neighbour experiment should compare SIS-like benchmark latency with and without a competing I/O or CPU workload. The mitigation experiment should measure the cost of host separation, resource limits, network policy or disabled SMT. If the measurements show that shared placement exceeds the 10% degradation target, the dedicated SIS pool is justified even if it uses more hosts. If Kata shows a measurable overhead compared with Docker, that cost is acceptable for external tenants when balanced against the stronger VM boundary and regulatory need to demonstrate separation.

The final architecture should be approved only after the measured results, host-capacity plan, migration security checks and tenant-isolation evidence are attached. The current assignment provides the analytical justification and a reproducible testing plan; actual experimental claims must be completed from the student's environment.

Conclusion

The analysis compares KVM/QEMU full virtualization, Docker OS-level containers and Kata micro-VMs. KVM provides a practical VM foundation for the university's core systems, Docker offers high density for trusted internal services, and Kata provides a stronger isolation boundary for external research workloads. The

quantitative model highlights the importance of CPU overcommit, memory reserve and shared network capacity. The security analysis identifies image control, patching, virtual-network segmentation and tenant isolation as essential safeguards.

For the changed EduCloud requirement, a hybrid architecture with separate sensitive, internal and external host pools is recommended. SIS and LMS workloads should remain on controlled KVM infrastructure, while external Spark/Hadoop jobs should run in Kata micro-VMs with strict quotas and isolated networks. Experimental measurements from Tasks 8–10 must be added before the design is treated as fully validated.

References and Evidence Notes

1. Assignment 2 handout: “Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud,” Cloud Computing and Big Data, CO2. The scenario, required tasks, host parameters and mark distribution in this report are based on the supplied handout.

2. KVM/QEMU, Docker and Kata Containers are the selected implementation families named or permitted by the assignment brief. Record the exact versions and configuration used during practical testing.

3. All numerical overhead and efficiency factors explicitly labelled as assumptions are modelling inputs for this assignment, not claims taken from an external benchmark.

Submission Checklist

- Add date and verify student details.
- Insert labelled architecture diagrams if your faculty requires drawn diagrams rather than the layered text diagrams in Task 1.
- Run the practical tests for Task 8 and attach commands/configuration files and screenshots.
- Replace all Task 9 placeholders with measured results from at least three repetitions.
- Create the two required graphs using actual measurements.
- Complete Task 10 using the measured values; do not leave prediction-error calculations blank.
- Attach evidence of at least one security property and one mitigation test.
- Check page numbers, spelling and formatting before submission.